arXiv:2410.21272v2 [cs.CL] 20 May 2025

ARITHMETIC WITHOUT ALGORITHMS: LANGUAGE MODELS SOLVE MATH WITH A BAG OF HEURISTICS

Yaniv Nikankin[1]* **Anja Reusch**[1] **Aaron Mueller**[1,2] **Yonatan Belinkov**[1]
[1]Technion – Israel Institute of Technology [2]Northeastern University

ABSTRACT

Do large language models (LLMs) solve reasoning tasks by learning robust generalizable algorithms, or do they memorize training data? To investigate this question, we use arithmetic reasoning as a representative task. Using causal analysis, we identify a subset of the model (a circuit) that explains most of the model's behavior for basic arithmetic logic and examine its functionality. By zooming in on the level of individual circuit neurons, we discover a sparse set of important neurons that implement simple heuristics. Each heuristic identifies a numerical input pattern and outputs corresponding answers. We hypothesize that the combination of these heuristic neurons is the mechanism used to produce correct arithmetic answers. To test this, we categorize each neuron into several heuristic types—such as neurons that activate when an operand falls within a certain range—and find that the unordered combination of these heuristic types is the mechanism that explains most of the model's accuracy on arithmetic prompts. Finally, we demonstrate that this mechanism appears as the main source of arithmetic accuracy early in training. Overall, our experimental results across several LLMs show that LLMs perform arithmetic using neither robust algorithms nor memorization; rather, they rely on a "*bag of heuristics*". [1]

1 INTRODUCTION

Do large language models (LLMs) implement robust reusable algorithms to solve tasks, or are they merely memorizing aspects of the training distribution? This distinction is crucial (Tänzer et al., 2022; Henighan et al., 2023): while memorization might suffice for limited problem sets, true algorithmic comprehension allows for generalization and efficient scaling to new problems.

Arithmetic reasoning provides a lens for this investigation, as it can be solved using various methods: learning known algorithms, developing novel approaches, or by memorizing vast quantities of input-output pairs. Thus, we ask the following: Do LLMs implement robust algorithms to correctly complete arithmetic prompts, similar to children learning vertical addition to add two numbers, or do LLMs merely memorize the arithmetic prompts that appear in their vast training data?

Previous studies have made progress in identifying arithmetic mechanisms in LLMs. Stolfo et al. (2023) and Zhang et al. (2024) have identified a subset of model components (*a circuit*) responsible for arithmetic calculations in several LLMs and characterized the information flow between them. Zhou et al. (2024) suggested that pre-trained LLMs use features in Fourier space to accurately answer addition prompts. However, Stolfo et al. (2023) and Zhang et al. (2024) stopped short of elucidating the mechanism implemented by the circuit they identified—a required feat to understand the trade-off between generalization and memorization in this task. Zhou et al. (2024) studied addition prompts in models that were *fine-tuned* on arithmetic data. Their findings regarding Fourier features are significant, but we claim these represent only a part of a more complex mechanism. Our work aims to bridge these gaps: we investigate how the arithmetic circuit works qualitatively—and specifically, whether it implements a mathematical algorithm or memorizes arithmetic training data.

To do so, we reverse-engineer the arithmetic mechanism applied by LLMs. We use causal analysis to examine their arithmetic circuits, focusing on individual neurons within the circuit responsible

*Correspondence to: yaniv.n@cs.technion.ac.il
[1]Code at https://github.com/technion-cs-nlp/llm-arithmetic-heuristics

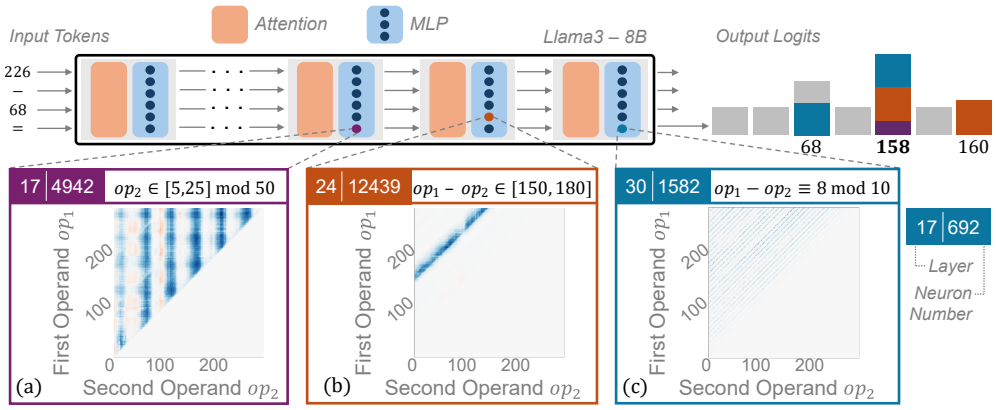


Figure 1: **Bag of heuristics visualization.** We show that transformer LLMs solve arithmetic prompts by combining several unrelated heuristics, each activating according to rules based on the input values of operands, and boosting the logits of corresponding result tokens. These heuristics are manifested in single MLP neurons in mid to late layers.

for generating the correct answer. Our analysis reveals that a sparse subset of neurons is sufficient for accurate responses, with each neuron implementing a distinct *heuristic*. Each heuristic fires for a specific pattern in input operands or in their combination, and some increase the logits of relevant result tokens accordingly. For instance, one heuristic (Figure 1b) increases logits of tokens between 150 and 180 for subtraction prompts whose answer falls within this range. By examining these neurons, we classify each into one or more heuristic types. For example, the mentioned neuron (Figure 1b) falls within the type of result-range heuristics, which promote continuous value ranges. We discover that successful prompt completion relies on a combination of several unrelated heuristic types, forming a *"bag of heuristics"* approach. This finding suggests that LLMs may not be employing a single, cohesive algorithm for arithmetic reasoning, nor are they memorizing all possible inputs and outputs; rather, they deploy a collection of simpler rules and patterns.

We investigate if the bag of heuristics emerges as the primary arithmetic mechanism from the onset of training, or whether it overrides an earlier mechanism. To do that, we analyze how the heuristics evolve over the course of training. We show arithmetic heuristics appear throughout the model training, gradually converging towards the heuristics observed in the final checkpoint. Furthermore, we provide evidence that the bag of heuristics mechanism explains most of the model's behavior even in early stages, indicating it is the main mechanism used for solving arithmetic prompts.

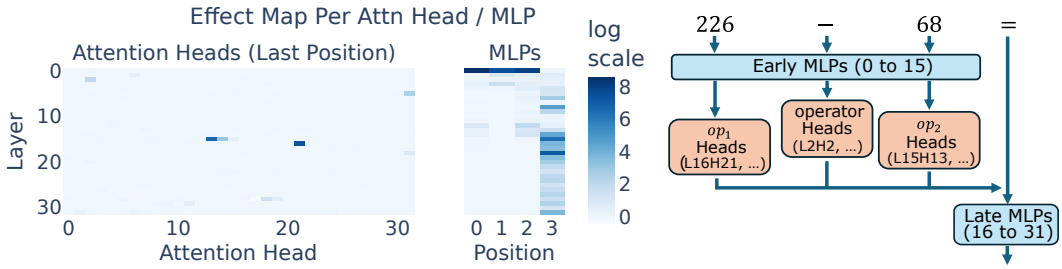
We contribute by providing a high-resolution understanding of the mechanism that LLMs use to answer arithmetic prompts. We (i) show pre-trained LLMs implement a "bag of heuristics" approach, (ii) investigate when and why this mechanism fails to generalize, and (iii) discover how it emerges across training. This allows us to better understand the source of current capabilities and limitations of LLMs in arithmetic reasoning—a finding that could apply to additional reasoning tasks.

2 Arithmetic Circuit Discovery

In transformer-based LLMs, a **circuit** (Elhage et al., 2021) refers to a minimal subset of interconnected model components (multi-layer perceptrons (MLP) or attention heads) that perform the computations required for a specific task. We locate, and later analyze, the circuit responsible for arithmetic calculations.

2.1 Circuit Discovery and Evaluation

Models and Data We analyze four LLMs: Llama3-8B/70B (Dubey et al., 2024), Pythia-6.9B (Biderman et al., 2023), and GPT-J (Wang & Komatsuzaki, 2021). For each, we locate and analyze the circuit responsible for arithmetic calculations. We focus on Llama3-8B in the main paper and report similar results for the additional models in Appendix I. We use pre-trained models without fine-tuning them on arithmetic prompts, as our goal is to uncover the mechanisms induced by typical



(a) (b)

Figure 2: **Llama3-8B arithmetic circuit discovery results. (a):** Few attention heads have a high effect on arithmetic prompts. Most MLPs take part in the computation. The first MLP noticeably affects operand and operator positions, while mid- and late-layer MLPs influence the final position. **(b):** The arithmetic circuit in Llama3-8B. The attention heads project token information to the last position, where the middle- and late-layer MLPs promote the logits for the correct answer.

language model training. Each model tokenizes positive numbers as a single token, up to some limit; e.g., in Llama3-8B, numbers in $[0, 1000]$ are tokenized to a single token. To locate the arithmetic circuit, we use two-operand arithmetic prompts with Arabic numerals and the four basic operators $(+, -, \times, \div)$, such that each prompt has four tokens: $op1$, the operator, $op2$, and the "=" sign. We sample a list of 100 prompts per operator for circuit discovery, and an identical amount for evaluation. Each prompt is chosen so that both its operands and result will be tokenized to a single token; e.g., in Llama3-8B, the operands and result must be between 0 and $1,000$. Unlike previous studies (Stolfo et al., 2023), we do not use in-context prompting, to ensure the circuit does not include any component not directly linked to arithmetic calculations. To reduce noise and ensure the circuits only contain components responsible for correct arithmetic completions, we only use prompts that are correctly completed by the model, similar to previous studies (Wang et al., 2022; Prakash et al., 2024). Throughout the paper, the prompt "$226 - 68 =$" is used as a running example.

Method To locate the circuit components, we conduct a series of activation patching experiments (Vig et al., 2020) that allow us to assess the importance of each MLP and attention head at each sequence position. Each experiment involves sampling a prompt p with result r from the dataset (for example, "$226 - 68 =$"), and sampling a random counterfactual prompt p' that leads to a different result r' (for example, "$21 + 17 =$"). After pre-computing the activations of the model for the counterfactual prompt p', we introduce the prompt p to the model. We intervene on (*"patch"*) the computation, which means we replace the activation of a single MLP layer or attention head with its pre-computed activation for p'. Following Stolfo et al. (2023), we observe how this intervention affects the probabilities of both answer tokens, r and r', by measuring the following:

$$\mathrm{E}(\mathrm{r}, \mathrm{r}') = \frac{1}{2}\left[\frac{\mathbb{P}^*(r') - \mathbb{P}(r')}{\mathbb{P}(r')} + \frac{\mathbb{P}(r) - \mathbb{P}^*(r)}{\mathbb{P}^*(r)}\right] \tag{1}$$

where $\mathbb{P}$ and $\mathbb{P}^*$ are the pre- and post-intervention probability distributions, respectively. The two summands in Equation (1) increase if patching raises the probability of r' or decreases the probability of r, respectively. High effect for an intervention on a component indicates its high importance in prompt calculation. The effect is averaged across prompts and measured separately per component.

Results The patching results, shown in Figure 2a, reveal that the MLP layers affect the output probabilities more than the attention heads. The first MLP affects the representation at the operator and operand positions (see Appendix B.1), while middle- and late-layer MLPs exhibit a strong effect at the final position, likely reflecting their role in predicting the answer token in that position (further discussed in Section 2.2). Figure 2a also shows that very few attention heads are important to the circuit. Each such attention head copies information from a single position (either an operand or operator position) to the final position (see Appendix B.2). Figure 2b summarizes the information flow within the circuit, consistent in structure with prior work (Stolfo et al., 2023).

To evaluate the circuit $\mathbf{c}$, we measure its faithfulness (Wang et al., 2022), the proportion of the full model's behavior on arithmetic prompts that can be explained solely by the circuit. To measure